Plan de verificación: UART 16550

Portada

Campo	Valor
Proyecto	Verificación del UART 16550 con UVM
Integrante 1	Josué Campos Herrera
Integrante 2	Sergio Salazar Núñez
Integrante 3	Juan Pablo Elizondo Espinoza
Versión del plan	1.0
Fecha	05 de noviembre del 2026
Repositorio Git	https://github.com/pabloelizondo17/VFCI-Proyecto-2.git
Etiqueta Git	entrega1

1. Alcance

Campo	Valor
Versión del RTL verificada (commit o revisión SVN)	github.com/freecores/uart16550, commit 2b0ad80d0968f201f95506854ad769c8a44abe2b (22/11/2004). Solo el directorio rtl/verilog/.
Configuración de compilación (bus de 32 bits; variante de 8 bits sí/no)	-Bus de 32 bits (DATA_BUS_WIDTH_8 sin definir, dirección de 5 bits, registros Debug 1 y 2 presentes). -UART_HAS_BAUDRATE_OUTPUT sin definir. -Variante de 8 bits: no sin embargo, una vez finalizado para 32 bits el ambiente se intentará diseñar para soportarla cambiando solo el define y el objeto de configuración -Compilación verificada con VCS X-2025.06-SP1, el DUT imprime “Data bus width is 32. Debug Interface present”
Frecuencia de reloj en simulación y rango de DL soportado	-Reloj de 50 MHz ya que en la especificación se exige al menos 3,6864 MHz para llegar a 115 200 baudios. -DL al ser de 16 bits tiene un rango de 0 a 65535, cuando DL = 0 queda deshabilitada la transmición por ende, se verifica que no hay actividad, se recomienda un DL de 1 a 16 para que la velocidad de transmisión que se calcula con f_clk / (16 * DL) no sea muy baja.
Funcionalidad fuera de alcance y justificación	-Variante de bus de 8 bits y baud_o. -Síntesis, temporización y cruce de dominios de reloj reales (el RTL se verifica funcionalmente a nivel RTL) no se toma en cuenta como se implementaría el circuito en silicio o en una FPGA real.

2. Documentos de referencia

Documento	Versión	Enlace o ubicación
UART IP Core Specification (OpenCores, J. Gorban)	Rev. 0.6	doc/UART_spec.pdf en el repositorio
UART IP Core Specification, versión revisada con aclaraciones y lista de errores conocidos	Rev. 0.6.1	https://uart16550.readthedocs.io/en/latest/uart16550doc.html
Enunciado del proyecto UVM: UART 16550	16/10/2026	Sistema del curso

3. Interpretación de la especificación

ID	Tema	La especificación dice	El RTL hace	Comportamiento que modelamos	Justificación
AMB-01	Registro scratch en la dirección 7	No existe: “The scratch register is removed, as it serves no purpose”. La tabla de registros (4.1 Registers list) no lista la dirección 7.	SI existe: reg [7:0] scratch en uart_regs.v se puede hacer operaciones de escritura y lectura en UART_REG_SR (macro de dirección 7) y reset a 0.	Registro de lectura y escritura (RW) de 8 bits en la dirección 7, reset 0, tal como lo implementa el RTL.	Es una discrepancia de documentación sin impacto funcional. Modelarlo permite ejercitar escritura y lectura en el RAL (reset y bit bash). No es un error del DUT nos sirve para confirmar que el UART sigue funcionando en caso de que el procesador quiera guardar algo ahí.
AMB-02	Lectura de vuelta del MCR (Modem Control Register)	MCR es de solo escritura (W) en la tabla de registros (4.1 Registers list).	El multiplexor de lectura no tiene caso para la dirección 4, siempre devuelve 8'b0.	MCR no legible: las lecturas por puerta delantera devuelven 0. El valor se comprueba por puerta trasera (uart_top.regs.mcr). Campo WO en el RAL.	RTL y especificación coinciden.
AMB-03	Polaridad de rts_pad_o y dtr_pad_o	Bit 0 del MCR en 1 hace DTR = 0 y bit 1 en 1 hace RTS = 0 (salidas invertidas respecto a los bits).	rts_pad_o = mcr[1] y dtr_pad_o = mcr[0] en uart_regs.v, sin inversión.	-Modo RTL_QUIRKS (por defecto): la salida sigue el RTL (sin inversión) y se cuenta como defecto D-01. -Con +UART_ENV_QUIRKS=0: sigue el manual y el scoreboard marca error.	El RTL contradice la especificación. Es un error del DUT, el ambiente por defecto mantiene la regresión en 0 errores sin esconder esa discrepancia.
AMB-04	Polaridad de MSR[7:4] respecto a las entradas de módem	MSR[7:4] es el complemento(la negación) de los bits CTS, DSR, RI y DCD. En loopback (cuando habla con sí mismo) es igual a RTS, DTR, OUT1 y OUT2.	En operación normal, MSR[7:4] toma cts_pad_i, dsr_pad_i, ri_pad_i y dcd_pad_i sin complementar. En loopback toma MCR[1], [0], [2] y [3], igual que la especificación.	-Loopback: según especificación y RTL (coinciden). -Operación normal: modo RTL_QUIRKS (sin complemento) y se cuenta como D-02.	En operación normal el RTL contradice la especificación. En el modo loopback no hay conflicto.
AMB-05	Condición de activación de TERI (MSR bit 2)	TERI se activa cuando RI cambia de bajo a alto.	msr[3:0] acumula (nuevo xor anterior): TERI se activa con cualquier cambio de ri (ya sea de bajo a alto o de alto a bajo), igual que los otros deltas.	-Modo RTL_QUIRKS: cualquier cambio de ri activa TERI (D-03). -Con QUIRKS = 0: solo el flanco indicado por la especificación.	El RTL contradice la especificación: TERI se activa con cualquier cambio de ri y no solo con el flanco de subida (ítem 10 de la lista «Legacy Bugs» de la 0.6.1). Por defecto el modelo sigue al RTL para mantener la regresión limpia y la diferencia se cuenta como D-03. Se cubre con cg_modem: cruce cp_delta[TERI] × cp_ri_edge, que distingue ambos flancos.

ID	Tema	La especificación dice	El RTL hace	Comportamiento que modelamos	Justificación
AMB-06	Decodificación con wb_sel_i sin exactamente un bit activo (32 bits)	La especificación solo indica que wb_sel_i selecciona el byte útil. No define el caso de varios bits o ninguno.	-Para Lecturas: un bit activo devuelve el byte en su carril, 4'b1111 devuelve el registro de depuración (wb_dat32_o); cualquier otro valor devuelve 0. -Para Escritura y dirección interna: con valor no one-hot usa el byte 0 (default usa el carril [7:0]), por lo que la escritura sí se ejecuta.	Estímulo normal: solo sel one-hot (y 4'b1111 para Debug 1 y 2). Con +UART_WB_SEL_ILLEGAL_PCT se generan valores no válidos y el modelo predice el comportamiento del RTL.	La especificación no define qué ocurre con un wb_sel_i que no sea one-hot, así que no es un defecto del DUT sino un caso no especificado. El RTL responde distinto según el acceso: la lectura devuelve 0 y la escritura se ejecuta con el byte 0. El modelo reproduce ese comportamiento y se documenta en el reporte como observación, sin tratarlo como error.
AMB-07	Más de un bit de parada	TX genera 1 bit, o 1,5 con 5 bits de carácter y 2 en los demás casos. “The receiver always checks the first stop bit only”.	-Receptor: calcula rframing_error con un solo bit de parada. -Transmisor (s_send_stop, uart_transmitter.v): carga el contador con 13, 21 o 29 ticks de reloj 16x para 1, 1,5 y 2 bits de parada, lo que equivale a +½ y +1 bit respecto al caso base, coherente con LCR[2]. El ancho absoluto del bit se confirma midiéndolo en simulación.	-Receptor: verifica solo el primer bit de parada. -Transmisor: según especificación (1, 1,5 o 2). El monitor serie mide la duración real del bit de parada.	Si el ancho medido difiere de lo especificado, se reporta como D-04. Pendiente de confirmar con simulación.
AMB-08	Divisor DL: DL = 0, DL = 1 y orden de escritura	-DL = 0 (valor de reset) deshabilita toda la E/S serie. -El contador arranca al escribir el byte bajo, por lo que se recomienda escribir DLM primero y DLL al final. -La tasa máxima se indica como clk/32.	-El rtl confima que con DL=0 se deshabilita la transmisión -start_dlc en uart_regs.v se activa al escribir DLL concuerda con el manual. -Comportamiento con DL = 1 por confirmar en simulación.	-DL = 0: sin actividad en stx. DL de 1 a 65535 legal. -El escritor de DL escribe en cualquier orden (cobertura del orden). -El modelo parte de que la tasa vale f_clk/(16·DL) solo después de escribir DLL.	La cota clk/32 de la especificación sugiere que DL = 1 podría no ser funcional. Si se confirma, se documenta y se acota el rango por defecto a DL de 2 en adelante, con DL = 1 como caso dirigido.
AMB-09	FCR: bit 0 ignorado y nivel de disparo tras el reset	-FCR[0] “para apagar” se ignora (la FIFO siempre se encuentra activa). -Valor de reset 11000000b, o sea nivel de disparo de 14, se le avisa al procesador hasta tener 14 caracteres.	-fcr almacena solo [7:6] y su reset es 2'b11. -Las FIFOs están siempre habilitadas. Ambas concuerdan con la especificación.	FCR[0] sin efecto. Nivel de disparo tras el reset: 14. FCR[5:3] ignorados.	Coinciden. Se documenta porque el nivel por defecto tras el reset es 14, no 1, y afecta cuándo se activa la primera interrupción de recepción.
AMB-10	Deltas de MSR en modo loopback	Los bits delta se activan cuando cambia la línea correspondiente. No se aclara si en loopback siguen a las entradas externas o a los bits del MCR.	Los deltas se calculan con las entradas externas (cts_pad_i, etc.), no con los valores del MCR, aunque MSR[7:4] sí refleja correctamente el MCR en loopback.	Por confirmar en simulación. Hasta entonces el modelo sigue al RTL y la cobertura incluye el cruce loopback × delta.	La especificación no define el caso. Se anota como ambigüedad y se decide con evidencia de simulación.

4. Lista de características

ID	Característica	Fuente (sección de la especificación)	Prioridad (alta, media, baja)
F-WB-01	Ciclos Wishbone simples de lectura y escritura con wb_ack_o.	IO ports (WISHBONE interface signals)	Alta
F-WB-02	Selección de byte con wb_sel_i en el bus de 32 bits.	IO ports (WISHBONE interface signals)	Alta
F-WB-03	Registros Debug 1 y Debug 2 (direcciones 8 y 12, solo lectura, wb_sel_i = 4'b1111).	Registers: Debug 1, Debug 2	Media
F-LCR-01	Longitud de carácter de 5, 6, 7 y 8 bits.	LCR, bits 1:0	Alta
F-LCR-02	Bits de parada: 1, 1,5 (5 bits) y 2; el receptor verifica solo el primero.	LCR, bit 2	Alta
F-LCR-03	Paridad: deshabilitada, par, impar, stick 0 y stick 1.	LCR, bits 5:3	Alta
F-LCR-04	Control de break en TX (stx forzado a 0).	LCR, bit 6	Media
F-DIV-01	Divisor DL y tasa de baudios f_clk/(16·DL); DL = 0 deshabilita la E/S serie; acceso con DLAB = 1.	Divisor Latches; LCR bit 7; AMB-08	Alta
F-TX-01	Transmisión desde THR: bit de inicio, datos LSB primero, paridad y parada con el ancho de bit correcto.	Operation; THR	Alta
F-TX-02	FIFO de transmisión de 16 posiciones: llenado, vaciado, desbordamiento y limpieza con FCR[2].	FCR, bit 2; LSR, bits 5 y 6	Alta
F-RX-01	Recepción hacia RBR con muestreo en el centro del bit y sincronización de srx.	Operation; RBR	Alta
F-RX-02	FIFO de recepción de 16 posiciones con niveles de disparo 1, 4, 8 y 14 y limpieza con FCR[1].	FCR, bits 7:6 y 1	Alta
F-RX-03	Detección de error de paridad, error de trama y break (LSR[2], [3], [4], [7]).	Line Status Register	Alta
F-RX-04	Overrun de la FIFO de recepción (LSR[1]).	Line Status Register	Alta
F-RX-05	Tolerancia a desajuste de baudios y rechazo de glitches de la línea.	Operation; changelog	Media
F-LSR-01	Bits DR, TFE y TE de LSR y borrado de los bits de error al leer LSR.	Line Status Register	Alta
F-INT-01	Habilitación independiente de cada causa con IER[3:0]; int_o es nivel activo en alto.	IER	Alta
F-INT-02	Prioridad de IIR: RLS, luego RDA o timeout, luego THRE, luego módem; IIR[0] = 1 sin pendientes.	IIR	Alta
F-INT-03	Interrupción de timeout tras 4 tiempos de carácter sin actividad con al menos un carácter en la FIFO.	IIR (Timeout Indication)	Alta
F-INT-04	Interrupción THRE y su limpieza al escribir THR o leer IIR.	IIR (THRE)	Alta
F-INT-05	Interrupción RLS y su limpieza al leer LSR.	IIR (RLS)	Alta
F-INT-06	Interrupción RDA con nivel de disparo y su limpieza al bajar de ese nivel.	IIR (RDA); FCR	Alta
F-INT-07	Interrupción de módem y su limpieza al leer MSR.	IIR (Modem Status)	Media
F-MOD-01	Salidas rts_pad_o y dtr_pad_o controladas por MCR; OUT1 y OUT2.	MCR, bits 0 a 3	Media
F-MOD-02	Entradas de módem: MSR[7:4], deltas y TERI con borrado al leer MSR.	MSR (Modem Status Register)	Media

ID	Característica	Fuente (sección de la especificación)	Prioridad (alta, media, baja)
F-MOD-03	Modo loopback: stx en 1, conexiones DTR→DSR, RTS→CTS, OUT1→RI y OUT2→DCD, y TX hacia RX interno.	MCR, bit 4	Alta
F-REG-01	Valores de reset de todos los registros.	Registers (Reset Value) e Initialization	Alta
F-REG-02	Registro scratch (dirección 7).	Registers list (4.1): la especificación lo marca como eliminado	Baja
F-REG-03	Acceso por DLAB: direcciones 0 y 1 cambian de significado; LCR, MCR, LSR y MSR accesibles con DLAB = 1.	Registers; LCR bit 7	Alta
F-RST-01	Reset asíncrono en cualquier momento: limpia FIFOs y registros y deja el DUT utilizable.	Initialization	Media
F-TX-03	Línea stx en reposo (1) tras el reset y mientras no se transmite	IO ports (stx_pad_o); Initialization	Media
F-INT-08	Salida int_o en estado definido tras el reset	IO ports (int_o); Initialization	Media

5. Matriz de verificación

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
V-WB-01	F-WB-01	Protocolo Wishbone: señales estables sin ack, ack acotado, sin X	SVA	A-WB-01, A-WB-02, A-WB-03, A-WB-04; cg_regs.cp_access	+UART_WB_DELAY_MAX
V-WB-02	F-WB-02	Byte correcto en cada carril y comportamiento con sel no válido	SCOREBOARD, RAL	cg_regs.cp_sel x cp_addr	+UART_WB_SEL_ILLEGAL_PCT
V-WB-03	F-WB-03	Debug 1 y 2 reflejan LSR, IER, IIR, LCR, MSR, MCR, FCR y contadores	SCOREBOARD, RAL	cg_regs.cp_addr[8, 12]	+UART_REG_RANDOM_ACCESS_PCT
V-LCR-01	F-LCR-01	Datos TX y RX correctos con cada longitud	SCOREBOARD, SVA	A-SER-04; cg_line_cfg.cp_wls x cp_dir	+UART_LCR_WLS_WEIGHTS
V-LCR-02	F-LCR-02	Ancho del bit de parada generado; el receptor acepta con el primer bit de parada	SCOREBOARD, COBERTURA	cg_line_cfg.cp_stb x cp_wls	+UART_LCR_STB_WEIGHTS
V-LCR-03	F-LCR-03	Bit de paridad generado y verificado en los cinco modos	SCOREBOARD	cg_line_cfg.cp_parity x cp_dir	+UART_LCR_PARITY_WEIGHTS
V-LCR-04	F-LCR-04	stx permanece en 0 mientras LCR[6] = 1 y se recupera al soltarlo	SCOREBOARD, SVA	A-SER-05; cg_line_cfg.cp_bc	+UART_LCR_BC_PCT
V-DIV-01	F-DIV-01	Ancho de bit = 16-DL ciclos; DL=0 sin actividad; reconfiguración de DL	SCOREBOARD, SVA	A-SER-02, A-SER-06;	+UART_DL_MIN, +UART_DL_MAX_FAST, +UART_DL_MAX, +UART_DL_TAIL_PCT, +UART_DL_ZERO_PCT, +UART_DL_WR_ORDER_WEIGHTS
V-TX-01	F-TX-01	Cada carácter escrito en THR sale por	SCOREBOARD	cg_trafico.cp_tx_burst_len x cp_wls	+UART_TX_BURST_MIN,

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
		stx en orden y con formato correcto			+UART_TX_BURST_MAX, +UART_WB_TX_DELAY_MAX
V-TX-02	F-TX-02	Llenado a 16, escritura con la FIFO llena, vaciado y limpieza; LSR correctos	SCOREBOARD, COBERTURA	cg_fifo.cp_tf_level, cg_trafico.cp_tx_burst_len[OVERFLOW], cg_fifo.cp_fifo_clear	+UART_TX_BURST_MAX, +UART_FCR_CLR_PCT
V-RX-01	F-RX-01	El carácter recibido por srx sale por RBR sin pérdida ni duplicados	SCOREBOARD	cg_trafico.cp_rx_gap x cp_wls	+UART_SER_GAP_MAX, +UART_WB_RX_POLLING_DELAY_MAX
V-RX-02	F-RX-02	Orden FIFO, niveles de disparo y limpieza con FCR[1]	SCOREBOARD, COBERTURA	cg_fifo.cp_rf_level x cp_tl	+UART_FCR_TL_WEIGHTS, +UART_FCR_CLR_PCT
V-RX-03	F-RX-03	LSR[2], [3], [4] y [7] se activan con los errores inyectados	SCOREBOARD	cg_err.cp_pe, cp_fe, cp_break x cp_parity	+UART_SER_PARITY_ERR_PCT, +UART_SER_FRAME_ERR_PCT, +UART_SER_BREAK_PCT
V-RX-04	F-RX-04	Overrun: el carácter excedente se pierde, la FIFO queda intacta y LSR[1] se activa	SCOREBOARD	cg_err.cp_oe x cp_tl	+UART_SW_SERVICE_DELAY_MIN
V-RX-05	F-RX-05	Recepción correcta dentro de la tolerancia, error de trama fuera; glitches ignorados	SCOREBOARD	cg_err.cp_baud_dev_high, cp_glitch	+UART_SER_BAUD_DEVIATION_PCT, +UART_SER_BAUD_OUT_PCT, +UART_SER_GLITCH_PCT
V-LSR-01	F-LSR-01	Valor de LSR en cada lectura y borrado de errores al leer	SCOREBOARD, RAL	cg_regs.cp_lsr_read x cp_err_pending	+UART_REG_RANDOM_ACCESS_PCT
V-INT-01	F-INT-01	Cada bit de IER habilita o enmascara solo su causa	SCOREBOARD, SVA, COBERTURA	A-INT-01; cg_int.cp_ier (16 combinaciones)	+UART_IER_BIT_PCT
V-INT-02	F-INT-02	IIR muestra la causa de mayor prioridad con causas simultáneas	SCOREBOARD	cg_int.cp_iir_id x cp_pending_set	+UART_IER_BIT_PCT, +UART_SER_PARITY_ERR_PCT, +UART_MODEM_TOGGLE_PCT
V-INT-03	F-INT-03	IIR[3:0] = 4'hC con datos bajo el nivel de disparo y sin actividad (Timeout)	SCOREBOARD	cg_int.cp_iir_id[TIMEOUT] x cp_tl	+UART_SW_SERVICE_DELAY_MAX
V-INT-04	F-INT-04	THRE se activa con la FIFO vacía y se limpia al escribir THR o leer IIR	SCOREBOARD	cg_int.cp_clear_by[THR_WRITE, IIR_READ]	+UART_SW_STYLE_WEIGHTS
V-INT-05	F-INT-05	RLS se activa con errores y se limpia al leer LSR	SCOREBOARD	cg_int.cp_clear_by[LSR_READ]	+UART_SER_PARITY_ERR_PCT
V-INT-06	F-INT-06	RDA se activa al alcanzar el nivel de disparo y se limpia al bajar	SCOREBOARD	cg_int.cp_iir_id[RDA] x cp_tl	+UART_FCR_TL_WEIGHTS
V-INT-07	F-INT-07	Interrupción de módem con los deltas y su limpieza al leer MSR	SCOREBOARD	cg_int.cp_clear_by[MSR_READ]; cg_modem.cp_delta	+UART_MODEM_TOGGLE_PCT
V-MOD-01	F-MOD-01	Salidas rts_pad_o y dtr_pad_o contra	SCOREBOARD, SVA	A-MOD-01; cg_modem.cp_mcr_bits	+UART_MCR_BIT_PCT,

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
		MCR (según modo de polaridad)			+UART_ENV_QUIRKS
V-MOD-02	F-MOD-02	MSR[7:4], deltas y TERI contra el modelo; borrado al leer	SCOREBOARD	cg_modem.cp_delta x cp_ri_edge	+UART_MODEM_TOGGLE_PCT, +UART_ENV_QUIRKS
V-MOD-03	F-MOD-03	En loopback: stx = 1, conexiones cruzadas y datos TX recibidos por RX	SCOREBOARD, SVA	A-SER-03, A-MOD-02; cg_modem.cp_loopback_active x cp_wls	+UART_MCR_LOOPBACK_WEIGHTS
V-REG-01	F-REG-01	Valores de reset por puertas delantera y trasera	RAL	uvm_reg_hw_reset_seq sin errores; cg_regs.cp_addr	+UART_RST_ASYNC_PCT
V-REG-02	F-REG-02	Escritura y lectura del scratch	RAL	cg_regs.cp_addr[7]	+UART_REG_RANDOM_ACCESS_PCT
V-REG-03	F-REG-03	Cambio de significado de direcciones 0 y 1 con DLAB	SCOREBOARD, RAL	cg_regs.cp_addr x cp_dlab x cp_rw	+UART_REG_RANDOM_ACCESS_PCT, +UART_REG_RECONFIG_PCT
V-RST-01	F-RST-01	Reset en medio de tráfico: FIFOs vacías, IIR = C1 y el DUT sigue operando	SCOREBOARD	cg_rst.cp_during	+UART_RST_ASYNC_PCT
V-SER-01	F-TX-03	Salida del puerto stx_pad_o en 1, luego de un reset o inactividad.	SVA	A-SER-01	+UART_RST_ASYNC_PCT, +UART_SER_GAP_MAX
V-INT-08	F-INT-08	La salida de interrupciones (int_o) queda en un estado definido tras el reset.	SVA	A-INT-02	+UART_RST_ASYNC_PCT

6. Estrategia de estímulo: prueba base

6.1 Espacio de aleatorización

Variable aleatoria	Objeto de configuración	Rango legal	Distribución por defecto	Knobs
lcr.wls (Word length Select)	uart_virtual_seqr_cfg	0 a 3 (5 a 8 bits)	Uniforme	wls_w[4]
lcr.stb (stop bits)	uart_virtual_seqr_cfg	0 o 1 (1 bit, o 1,5 y 2 bits)	Uniforme	stb_w[2]
lcr.pen, eps, sp (modo de paridad)	uart_virtual_seqr_cfg	ninguna, par, impar, stick 0, stick 1	ninguna 2/6, resto 1/6 cada uno	parity_w[5]
lcr.bc (break TX)	uart_virtual_seqr_cfg	0 o 1 durante un intervalo	2%	bc_pct
dl	uart_virtual_seqr_cfg	0 a 65535	1% en 0, 97% en 1 a 16, 2% en 17 a 65535	dl_zero_pct, dl_min, dl_max_fast, dl_max, dl_tail_pct
Orden de escritura de DL	uart_virtual_seqr_cfg	DLM primero o DLL primero	75% DLM primero, 25% DLL primero	dl_wr_order_w[2]
fcr.tl, fcr.rx_clr, fcr.tx_clr	uart_virtual_seqr_cfg	1, 4, 8, 14; limpieza 0 o 1	TL uniforme; limpieza 10%	tl_w[4], clr_pct

Variable aleatoria	Objeto de configuración	Rango legal	Distribución por defecto	Knobs
ier[3:0]	uart_virtual_seqr_cfg	0 a 15	Cada bit 50% independiente	ier_bit_pct[4]
mcr.loopback	uart_virtual_seqr_cfg	0 o 1	85% apagado, 15% encendido	loopback_w[2]
mcr.dtr, rts, out1, out2	uart_virtual_seqr_cfg	0 o 1 cada uno	50% cada uno	mcr_bit_pct[4]
Longitud de ráfaga TX	uart_virtual_seqr_cfg	1 a 64 escrituras seguidas a THR	Uniforme entre 1 y 20	tx_burst_min, tx_burst_max
Error de paridad, trama y break en RX	serial_agent_cfg	0 a 100% de caracteres	5%, 5% y 2%	parity_err_pct, frame_err_pct, break_pct
Glitch en la línea	serial_agent_cfg	0 a 100%	2%	glitch_pct
Desviación de baudios del agente serie	serial_agent_cfg	0 a 10% de la tasa	Hasta 2% dentro de tolerancia; 10% fuera de tolerancia	baud_deviation_pct, baud_out_pct
Separación entre caracteres RX	serial_agent_cfg	0 a 200 tiempos de bit	Uniforme entre 0 y 20	gap_max
Cambios en cts, dsr, ri y dcd	mod_agent_cfg	Probabilidad de cambio por intervalo	10% por intervalo; retardo hasta 1000 ciclos	toggle_pct, toggle_delay_max
Estilo de servicio del software	uart_virtual_seqr_cfg	interrupciones o sondeo	50% y 50%	style_w[2]
Retraso de servicio	uart_virtual_seqr_cfg	0 a 100 tiempos de carácter	Uniforme entre 0 y 40	service_delay_min, service_delay_max
Accesos aleatorios a registros y reconfiguración	uart_virtual_seqr_cfg	0 a 100%	5% accesos extra, 10% reconfiguración	reg_access_pct, reconfig_pct
wb_sel_i no válido	wb_agent_cfg	0 a 100% de los accesos	1%	sel_illegal_pct
Retardo entre transacciones Wishbone	wb_agent_cfg	0 a 10 ciclos	Uniforme entre 0 y 3	tx_delay_max, rx_polling_delay_max, delay_max
Reset asíncrono	uart_env_cfg	0 a 100% por época	1%	rst_async_pct

6.2 Argumento de completitud

Bin o grupo de bins	Por qué su probabilidad es mayor que cero con los valores por defecto	Plusarg que lo acelera
cg_err.cp_oe × cp_tl (desbordamiento de memoria)	La FIFO RX se llena si el retraso de servicio supera el tiempo de 16 caracteres; el rango por defecto (hasta 40 tiempos de carácter) lo permite y cada nivel de disparo tiene probabilidad mayor que cero.	+UART_SW_SERVICE_DELAY_MIN
cg_int.cp_iir_id[TIMEOUT] (interrupción por timeout)	Con el servicio retrasado y pocos caracteres recibidos, el retraso supera 4 tiempos de carácter y el nivel de disparo no se alcanza; ambas condiciones tienen probabilidad mayor que cero.	+UART_SW_SERVICE_DELAY_MAX, +UART_FCR_TL_WEIGHTS
cg_err.cp_pe, cp_fe, cp_break × cp_parity (errores de	Los errores se inyectan con porcentajes por defecto mayores	+UART_SER_PARITY_ERR_PCT,

Bin o grupo de bins	Por qué su probabilidad es mayor que cero con los valores por defecto	Plusarg que lo acelera
paridad, trama y break por modo de paridad)	que cero y el modo de paridad es independiente del error.	+UART_SER_FRAME_ERR_PCT, +UART_SER_BREAK_PCT
cg_err.cp_baud_dev_high (desviación de baudios fuera de tolerancia)	Un 10 % de los caracteres usan una desviación mayor que 2 % y hasta 4 % por defecto, por encima de la tolerancia de 2,4 % de la especificación.	+UART_SER_BAUD_OUT_PCT
cg_line_cfg.cp_dl[tail] (divisores grandes)	El 2 % de las épocas elige DL entre 17 y 65535. Con DL grande el tiempo de simulación sube mucho, por lo que se limita el número de caracteres en esas épocas.	+UART_DL_TAIL_PCT, +UART_DL_MAX
cg_line_cfg.cp_dl_wr_order (orden de escritura de DLL y DLM)	El orden de escritura de DL se elige en cada reconfiguración con pesos 75 y 25, así que ambos órdenes tienen probabilidad mayor que cero. Con DLL primero la secuencia espera la recarga del contador antes de mandar tráfico.	+UART_DL_WR_ORDER_WEIGHTS
cg_line_cfg.cp_dl_zero (divisor en cero)	El 1 % de las épocas usa DL = 0, caso en que se verifica ausencia de actividad.	+UART_DL_ZERO_PCT
cg_trafico.cp_tx_burst_len[OVERFLOW] (desbordamiento de la FIFO TX)	La ráfaga TX llega a 20 escrituras seguidas y la FIFO tiene 16 posiciones; con DL grande la FIFO no drena durante la ráfaga.	+UART_TX_BURST_MAX
cg_modem.cp_ri_edge, cp_loopback_active × cp_delta (cambios de módem y TERI, con y sin loopback)	Los cuatro cambios de módem son independientes del loopback, con 10 % de probabilidad por intervalo y loopback en 15 % de las épocas.	+UART_MODEM_TOGGLE_PCT, +UART_MCR_LOOPBACK_WEIGHTS
cg_rst.cp_during (reset asíncrono en medio del tráfico)	El reset asíncrono tiene 1 % por época y puede caer con TX o RX activos y FIFOs no vacías. Se justifica el valor bajo por el costo de reiniciar el modelo.	+UART_RST_ASYNC_PCT
cg_regs.cp_sel[illegal] (wb_sel_i no válido)	1 % de los accesos usa wb_sel_i distinto de one-hot y de 4'b1111.	+UART_WB_SEL_ILLEGAL_PCT

6.3 Secuencia virtual principal

```
task body();
  // Iteración de sub-pruebas (épocas) leyendo el plusarg
  repeat (n_epochs = +UART_SEQ_NUM_EPOCHS) begin

    // 1. Reset asíncrono aleatorio (Opcional por época)
    if (rst_async_pct) begin
      aplicar_reset_asincrono();
    end

    // 2. Aleatorización de la configuración en las transacciones
    epoch_cfg.randomize();

    // 3. Configuración del DUT (Frontdoor por el RAL)
    // Se escribe LCR (DLAB=1), divisores DLM/DLL, LCR final, FCR, IER, MCR
    programar_registros(epoch_cfg);

    // 4. Sincronización del reloj baud
    if (epoch_cfg.dl_wr_order == DLL_FIRST) begin
      esperar_recarga_contador();
    end

    // 5. Generación de tráfico concurrente
    fork
      tx_traffic();      // Ráfagas a THR para desbordar FIFO TX
      rx_traffic();      // Inyección serial con errores y variaciones
      modem_traffic();   // Fluctuación en pines CTS, DSR, RI, DCD
      sw_service(style); // Atención demorada para forzar overrun/timeout
      reg_noise();       // Accesos inválidos al bus (ruido)
    join

    // 6. Drenaje para cierre de la sub-prueba (época)
    esperar_tx_vacio();
    esperar_rx_leida();
  end
endtask
```

7. Tabla de plusargs

Plusarg	Tipo y rango	Valor por defecto	Componentes que lo usan	Objeto de configuración	Efecto
+UART_LCR_WLS_WEIGHTS	4 enteros >= 0	1,1,1,1	Driver WB y secuencias, scoreboard	uart_virtual_seqr_cfg	Pesos de la longitud de carácter
+UART_LCR_STB_WEIGHTS	2 enteros >= 0	1,1	Secuencias, scoreboard, monitor serie	uart_virtual_seqr_cfg	Pesos de bits de parada
+UART_LCR_PARITY_WEIGH	5 enteros >= 0	2,1,1,1,1	Secuencias, scoreboard,	uart_virtual_seqr_cfg	Pesos del modo de paridad

Plusarg	Tipo y rango	Valor por defecto	Componentes que lo usan	Objeto de configuración	Efecto
TS			driver serie		
+UART_LCR_BC_PCT	Entero 0 a 100	2	Secuencias, scoreboard	uart_virtual_seqr_cfg	Probabilidad de forzar break en TX
+UART_DL_MIN	Entero 0 a 65535	1	Secuencias de configuración	uart_virtual_seqr_cfg	Mínimo de DL
+UART_DL_MAX_FAST	Entero >= DL_MIN	16	Secuencias de configuración	uart_virtual_seqr_cfg	Tope del rango rápido de DL
+UART_DL_MAX	Entero <= 65535	65535	Secuencias de configuración	uart_virtual_seqr_cfg	Tope global de DL
+UART_DL_TAIL_PCT	Entero 0 a 100	2	Secuencias de configuración	uart_virtual_seqr_cfg	Probabilidad de elegir DL en la cola
+UART_DL_ZERO_PCT	Entero 0 a 100	1	Secuencias de conf., scoreboard	uart_virtual_seqr_cfg	Probabilidad de DL=0
+UART_DL_WR_ORDER_WEIGHTS	2 enteros >= 0	3,1	Secuencias de conf., scoreboard	uart_virtual_seqr_cfg	Pesos del orden de escritura DLL y DLM
+UART_FCR_TL_WEIGHTS	4 enteros >= 0	1,1,1,1	Secuencias, scoreboard	uart_virtual_seqr_cfg	Pesos del nivel de disparo
+UART_FCR_CLR_PCT	Entero 0 a 100	10	Secuencias, scoreboard	uart_virtual_seqr_cfg	Probabilidad de limpiar FIFOs al reconfigurar
+UART_IER_BIT_PCT	4 enteros 0 a 100	50,50,50,50	Secuencias, modelo interrupciones	uart_virtual_seqr_cfg	Probabilidad de habilitar cada bit de IER
+UART_MCR_LOOPBACK_WEIGHTS	2 enteros >= 0	85,15	Secuencias, scoreboard, SVA	uart_virtual_seqr_cfg	Pesos del modo loopback
+UART_MCR_BIT_PCT	4 enteros 0 a 100	50,50,50,50	Secuencias, monitor de módem	uart_virtual_seqr_cfg	Probabilidad de activar cada bit de MCR
+UART_TX_BURST_MIN	Entero >= 1	1	Secuencia TX	uart_virtual_seqr_cfg	Mínimo de escrituras seguidas a THR
+UART_TX_BURST_MAX	Entero a 64	20	Secuencia TX	uart_virtual_seqr_cfg	Máximo de escrituras seguidas a THR
+UART_SER_PARITY_ERR_PCT	Entero 0 a 100	5	Driver serie, scoreboard	serial_agent_cfg	Porcentaje de caracteres RX con paridad corrupta
+UART_SER_FRAME_ERR_PCT	Entero 0 a 100	5	Driver serie, scoreboard	serial_agent_cfg	Porcentaje de caracteres RX con error trama
+UART_SER_BREAK_PCT	Entero 0 a 100	2	Driver serie, scoreboard	serial_agent_cfg	Porcentaje de caracteres RX que son break
+UART_SER_GLITCH_PCT	Entero 0 a 100	2	Driver serie, scoreboard	serial_agent_cfg	Probabilidad de glitch en la línea
+UART_SER_BAUD_DEVIATION_PCT	Entero 0 a 10	4	Driver serie	serial_agent_cfg	Desviación máxima de baudios del emisor
+UART_SER_BAUD_OUT_PCT	Entero 0 a 100	10	Driver serie, scoreboard	serial_agent_cfg	Porcentaje caracteres fuera de tolerancia
+UART_SER_GAP_MAX	Entero >= 0	20	Driver serie	serial_agent_cfg	Hueco máximo entre caracteres RX

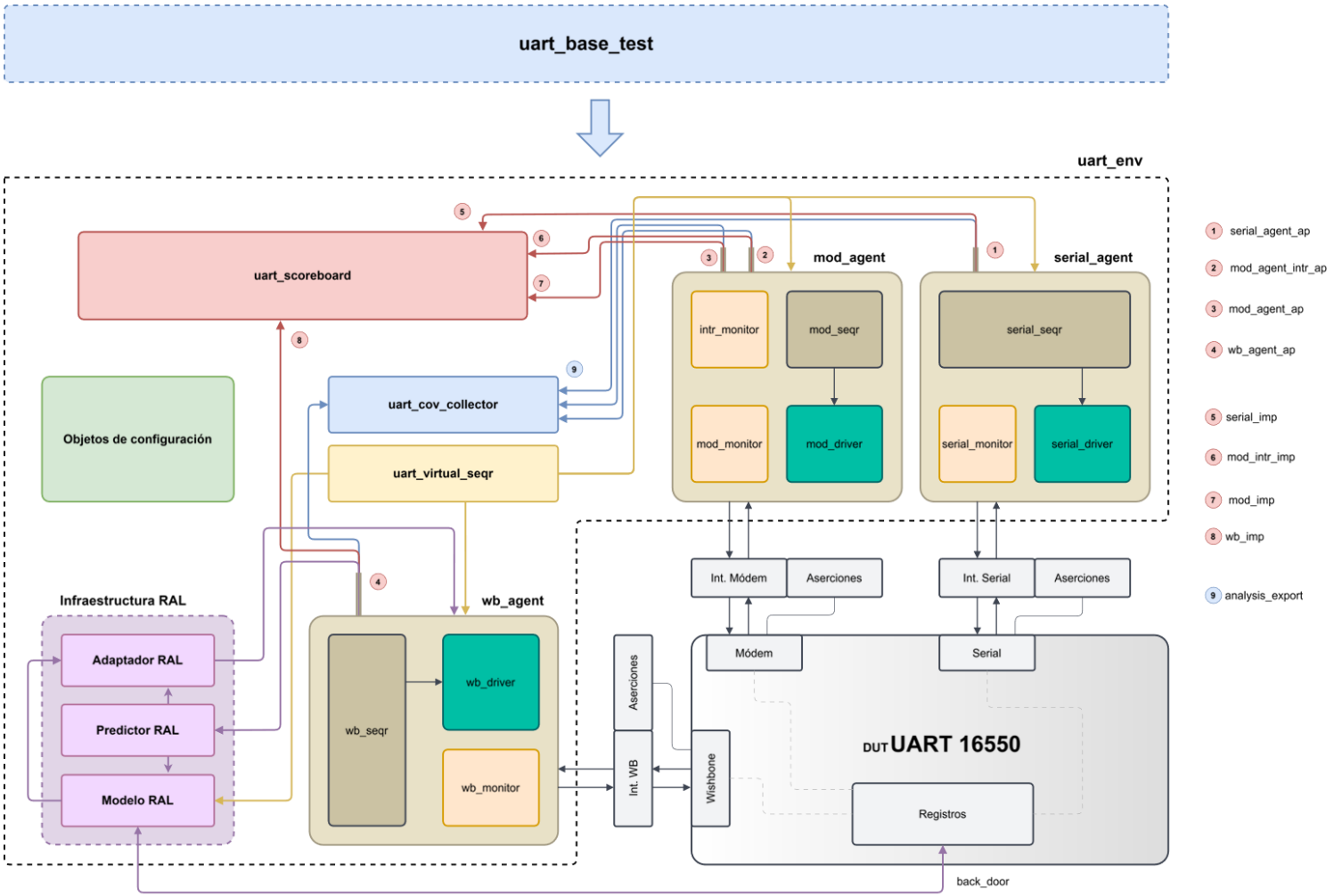
Plusarg	Tipo y rango	Valor por defecto	Componentes que lo usan	Objeto de configuración	Efecto
+UART_MODEM_TOGGLE_PCT	Entero 0 a 100	10	Driver módem, scoreboard	mod_agent_cfg	Probabilidad de cambio señales de módem
+UART_MODEM_TOGGLE_DELAY_MAX	Entero >= 0	1000	Driver módem	mod_agent_cfg	Retardo máximo entre cambios de módem
+UART_SW_STYLE_WEIGHTS	2 enteros >= 0	1,1	Secuencia de servicio	uart_virtual_seqr_cfg	Estilo de atención del software
+UART_SW_SERVICE_DELAY_MIN	Entero >= 0	0	Secuencia de servicio	uart_virtual_seqr_cfg	Retraso mínimo antes de atender
+UART_SW_SERVICE_DELAY_MAX	Entero >= 0	40	Secuencia de servicio	uart_virtual_seqr_cfg	Retraso máximo antes de atender
+UART_REG_RANDOM_ACCESS_PCT	Entero 0 a 100	5	Secuencia de ruido de registros	uart_virtual_seqr_cfg	Accesos extra a cualquier dirección
+UART_REG_RECONFIG_PCT	Entero 0 a 100	10	Secuencia principal	uart_virtual_seqr_cfg	Reconfiguración en medio del tráfico
+UART_WB_SEL_ILLEGAL_PCT	Entero 0 a 100	1	Driver WB, scoreboard	wb_agent_cfg	Accesos con wb_sel_i no válido
+UART_WB_DELAY_MAX	Entero >= 0	3	Driver WB	wb_agent_cfg	Retardo máximo entre transacciones WB
+UART_WB_TX_DELAY_MAX	Entero >= 0	3	Driver WB, secuencia TX	wb_agent_cfg	Retardo máximo escrituras THR
+UART_WB_RX_POLLING_DELAY_MAX	Entero >= 0	3	Driver WB, secuencia de servicio	wb_agent_cfg	Retardo máximo lecturas LSR/RBR
+UART_RST_ASYNC_PCT	Entero 0 a 100	1	Secuencia principal, scoreboard	uart_env_cfg	Probabilidad de reset asíncrono
+UART_SEQ_NUM_EPOCHS	Entero >= 1	10	Secuencia principal	uart_virtual_seqr_cfg	Número de épocas por simulación
+UART_ENV_QUIRKS	0 o 1	1	Scoreboard, monitores módem	uart_scoreboard_cfg	Sigue al RTL en discrepancias (1) o manual (0)

8. Casos de esquina planificados

ID	Escenario	Archivo .args	Plusargs	Bins objetivo	Semillas
CE-01	Overrun de la FIFO RX con niveles de disparo 14, 8, 4 y 1.	rx_overrun_rand_tl.args	+UART_FCR_TL_WEIGHTS=1, 1, 1, 1 +UART_SW_SERVICE_DELAY_MIN=5000	cg_err.cp_oe × cp_tl	60
CE-02	Overrun de la FIFO TX.	tx_overrun.zargs	+UART_TX_BURST_MIN=20 +UART_TX_BURST_MAX=50 +UART_WB_TX_DELAY_MAX=0	cg_trafico.cp_tx_burst_len[OVERFLOW]	20
CE-03	Framing Error y Break Interrupt en la línea srx_pad_i.	rx_break_condition.args	+UART_SER_BREAK_PCT=100 +UART_LCR_WLS_WEIGHTS=0,0,0,1	cg_err.cp_break × cg_err.cp_fe	20
CE-04	Desviación en la tasa de baudios.	rx_baud_mismatch.args	+UART_SER_BAUD_DEVIATION_PCT=4 +UART_DL_MIN=1	cg_err.cp_baud_dev_high	20
CE-05	Ráfaga de interrupciones de módem en modo Loopback.	mod_loopback_mode.args	+UART_MCR_LOOPBACK_WEIGHTS=0,1 +UART_MODEM_TOGGLE_DELAY_MAX=0	cg_modem.cp_loopback_active × cg_int.cp_iir_id[MODEM_STATUS]	20
CE-06	Colisión de interrupciones con diferente prioridad.	int_priority_collision.args	+UART_SER_PARITY_ERR_PCT=100 +UART_FCR_TL_WEIGHTS=1,0,0,0	cg_int.cp_iir_id[RLS] × cg_int.cp_rx_data_ready_pending	20
CE-07	Lectura de la interfaz WB y escritura de la interfaz Serial, de forma simultánea.	rx_simultaneous_rw.args	+UART_WB_RX_POLLING_DELAY_MAX=0 +UART_DL_MIN=1 +UART_DL_MAX=2	cg_fifo.cp_rbr_read × cg_fifo.cp_rx_push	20
CE-08	Intento de transmisión con el divisor latch en cero.	tx_disabled_dl.args	+UART_DL_MIN=0 +UART_DL_MAX=0	g_line_cfg.cp_dl_zero × cg_trafico.cp_tx_write_attempt	20

9. Arquitectura del ambiente

9.1 Diagrama



9.2 Componentes

Componente	Clase UVM base	Responsabilidad	Puertos TLM
wb_agent (activo)	uvm_agent	Maneja y observa la interfaz Wishbone.	wb_agent_ap (uvm_analysis_port)
- wb_driver	uvm_driver	Conduce señales de forma activa a la interfaz Wishbone.	seq_item_port
- wb_monitor	uvm_monitor	Captura la actividad de la interfaz Wishbone.	wb_mon_ap (uvm_analysis_port)
- wb_seqr	uvm_sequencer	Genera transacciones de datos para el controlador de la interfaz Wishbone.	seq_item_export
serial_agent (activo y pasivo)	uvm_agent	Maneja y observa la interfaz Serial.	serial_agent_ap (uvm_analysis_port)
- serial_driver	uvm_driver	Conduce señales de forma activa a la interfaz Serial.	seq_item_port
- serial_monitor	uvm_monitor	Captura la actividad de la interfaz Serial.	serial_mon_ap (uvm_analysis_port)
- serial_seqr	uvm_sequencer	Genera transacciones de datos para el controlador de la interfaz Serial.	seq_item_export
mod_agent (activo)	uvm_agent	Maneja y observa la interfaz Módem.	mod_agent_ap (uvm_analysis_port) mod_agent_intr_ap (uvm_analysis_port)
- mod_driver	uvm_driver	Conduce señales de forma activa a la interfaz Módem.	seq_item_port
- mod_monitor	uvm_monitor	Captura la actividad de la interfaz Módem.	mod_mon_ap (uvm_analysis_port)
- intr_monitor	uvm_monitor	Captura la actividad del puerto de interrupción (int_o).	mod_intr_mon_ap (uvm_analysis_port)
- mod_seqr	uvm_sequencer	Genera transacciones de datos para el controlador de la interfaz Módem.	seq_item_export
uart_virtual_seqr	uvm_sequencer	Coordina los secuenciadores Wishbone, Serial y Módem.	N/A
uart_scoreboard	uvm_scoreboard	Evalúa el comportamiento del diseño comparando los estímulos inyectados (predicción), con las salidas reales.	wb_imp, serial_imp, mod_imp, mod_intr_imp (uvm_analysis_imp).
Modelo RAL	uvm_reg_block + uvm_reg	Mantener el mapa de memoria abstracto, el espejo de los registros del hardware y su valor deseado.	N/A.
Predictor RAL	uvm_reg_predictor	Escuchar pasivamente el tráfico del bus y mantener los valores del modelo RAL actualizados.	bus_in (uvm_analysis_imp)
Adaptador RAL	uvm_reg_adapter	Actuar como diccionario de traducción bidireccional; es decir, convierte comandos abstractos a transacciones físicas y traduce transacciones físicas	N/A

Componente	Clase UVM base	Responsabilidad	Puertos TLM
		a operaciones abstractas de registro.	
uart_cov_collector	uvm_subscriber	Muestrea los grupos de cobertura cada vez que recibe una transacción de un monitor.	analysis_export (uvm_analysis_export)
uart_env	uvm_env	Instancia todos los agentes, el RAL, el predictor y el scoreboard. Realiza todas las conexiones TLM en la fase de conexión.	N/A
uart_base_test	uvm_test	Instancia el entorno superior. Lee argumentos de consola (plusargs), configura knobs en el uvm_config_db y lanza la secuencia virtual principal.	N/A

9.3 Flujo de configuración

Objeto de configuración	Ruta en uvm_config_db	Lo publica	Lo obtienen
wb_agent_cfg	uvm_test_top.uart_env.wb_agent.*	uart_base_test	Driver, monitor y secuencias Wishbone.
serial_agent_cfg	uvm_test_top.uart_env.serial_agent.*	uart_base_test	Driver, monitor y secuencias Serial.
mod_agent_cfg	uvm_test_top.uart_env.mod_agent.*	uart_base_test	Driver, monitor y secuencias Módem.
uart_virtual_seqr_cfg	uvm_test_top.uart_env.uart_virtual_seqr	uart_base_test	El componente uart_virtual_seqr.
uart_scoreboard_cfg	uvm_test_top.uart_env.uart_scoreboard	uart_base_test	El componente uart_scoreboard.
uart_reg_block_cfg	uvm_test_top.uart_env.uart_virtual_seqr.*	uart_env	Secuencias virtuales y el predictor.
uart_cov_collector_cfg	uvm_test_top.uart_env.uart_cov_collector	uart_base_test	El componente uart_cov_collector.
uart_env_cfg	uvm_test_top.uart_env	uart_base_test	El componente uart_env.

10. Modelo de referencia y scoreboard

Qué se predice	Fuente de la predicción	Momento de la comparación	Tolerancia
Datos TX	Escrituras a THR (monitor Wishbone)	Al reconstruir el carácter en stx_pad_o	Ninguna.
Datos RX	Escrituras por srx_pad_i (monitor Serial).	Al leer el RBR por la interfaz Wishbone.	Ninguna.
LSR	Estado de las FIFOs y errores presentados (monitor Wishbone).	Justo después de que el monitor Wishbone reporte una lectura en la dirección del registro.	Ninguna.
IIR	Modelo de interrupciones con prioridades a leer del registro (monitor Wishbone).	Al activarse (en alto) int_o, y justo después de efectuar una lectura en la dirección del registro.	Ninguna. Varía según el tipo de interrupción y su prioridad.
MSR	Cambios en las líneas de estado módem y estados de las líneas de control de módem (monitor Wishbone).	Justo después de leer en la dirección del registro.	Ninguna. Debe reflejar los cambios y estados correctamente.

Qué se predice	Fuente de la predicción	Momento de la comparación	Tolerancia
int_o	Lectura del puerto de interrupciones (monitor Wishbone).	Al validar con el registro IIR y su modelo de interrupciones.	Ninguna. Varía según el tipo de interrupción y su prioridad.
Salidas (rts_pad_o y dtr_pad_o) de módem	Escrituras en el MCR (monitor Wishbone).	Al detectar y leer un cambio por el monitor Serial.	Ninguna.
Niveles de FIFOs	Modelo de FIFOs (en software), contra Debug 2 o backdoor.	Luego de cada transacción y al terminar la prueba.	Ninguna. Deben de concordar en cantidad.
Tasa de Baudios	Registros DLL y DLM, en conjunto con la frecuencia de reloj del sistema (monitor Wishbone).	Al medir la duración de los bits capturados por el monitor serial en TX.	Baja (pocos ciclos de reloj).
Modo Loopback	Bit 4 del registro MCR (monitor Wishbone).	Tras escribir en MCR y mientras el modo esté activo.	Ninguna.
Estado tras reset	Al aplicar un reset (monitor Wishbone).	Luego de cada reset aplicado.	Ninguna.

11. Aserciones

ID	Interfaz	Propiedad (en palabras)	Tipo (assert, assume, cover)	Ítem del plan
A-WB-01	Wishbone	adr, dat_i, we y sel estables mientras cyc y stb están activos y no hay ack.	assert y cover	V-WB-01
A-WB-02	Wishbone	ack se activa tras un número acotado de ciclos de stb y cyc.	assert y cover	V-WB-01
A-WB-03	Wishbone	ack no se activa mientras que no se hayan presentado de previo, cyc y stb.	assert	V-WB-01
A-WB-04	Wishbone	ack no debe de estar en un estado indefinido tras un reset o en su operación normal.	assert	V-WB-01
A-SER-01	Serial	Salida del puerto stx_pad_o en 1, luego de un reset o cuando no se está transmitiendo.	assert	V-SER-01
A-SER-02	Serial	Cada bit transmitido por stx_pad_o, dura un tiempo físico de 16·DL ciclos de reloj.	assert y cover	V-DIV-01
A-SER-03	Serial	En el modo loopback, el pin físico stx_pad_o permanece en alto.	assert y cover	V-MOD-03
A-SER-04	Serial	Cada bit de inicio por el puerto serial, va seguido de la cantidad de bits configurada en el LCR.	assert	V-LCR-01 (Parcial)
A-SER-05	Serial	Con el bit 6 del LCR en 1, la salida serial (stx) se mantiene en 0 (break state).	assert y cover	V-LCR-04
A-SER-06	Serial	Con un valor de cero en DL, el periférico no transmite y por lo tanto la salida serial (stx) se mantiene en alto siempre, sin transmitir datos.	assert y cover	V-DIV-01 (Parcial)

ID	Interfaz	Propiedad (en palabras)	Tipo (assert, assume, cover)	Ítem del plan
A-MOD-01	Módem	rts_pad_o y dtr_pad_o reflejan correctamente lo especificado desde el MCR.	assert y cover	V-MOD-01
A-MOD-02	Módem	En el modo loopback los espacios del MSR[7:4] igualan a los del MCR[3:0].	assert y cover	V-MOD-03
A-INT-01	Módem	Con todos los bits de IER en cero, la salida de interrupciones (int_o) debe permanecer en cero.	assert y cover	V-INT-01 (Parcial)
A-INT-02	Módem	La salida de interrupciones (int_o), debe de quedar en un estado definido tras el reset.	assert	V-INT-08

12. RAL

12.1 Registros y campos

Registro	Dirección	DLAB	Campo	Bits	Acceso	Reset	Ruta de puerta trasera	Función
LCR	3	0 y 1	BITS	1:0	RW	2'b11	uart_top.regs.lcr[1:0]	Cuantos bits de datos por carácter (00=5, 01=6, 10=7, 11=8)
			SB	2	RW	1’b0	uart_top.regs.lcr[2]	Bits de parada (0=uno, 1=dos) o 1.5 si la palabra es de 5 bits. Espacio entre caracteres
			PE	3	RW	1’b0	uart_top.regs.lcr[3]	Habilita paridad, se genera al transmitir y verifica al recibir
			EP	4	RW	1’b0	uart_top.regs.lcr[4]	Elige paridad par (1) o impar (0)
			SP	5	RW	1’b0	uart_top.regs.lcr[5]	Paridad fija con PE=1, transmite y verifica el bit como constante en lugar de calcularlo
			BC	6	RW	1’b0	uart_top.regs.lcr[6]	Fuerza la salida serial a cero (condición ruptura)
			DL	7	RW	1’b0	uart_top.regs.lcr[7]	Conmuta las direcciones 0 y 1 hacia el divisor
RBR	0	0	RX	7:0	RO	-	uart_top.regs.rf_data_out[10:3]	Ventana de salida de la cola de recepción. Entrega el carácter más viejo.
THR	0	0	TX	7:0	WO	-	N/A (Escribe en la FIFO TX)	Ventana de entrada de la cola de transmisión. Escribir aquí encola un byte y no es leíble.
IER	1	0	RDA	0	RW	1’b0	uart_top.regs.ier[0]	Habilita “datos disponibles” y también el timeout. Cuando la cola de recepción alcanza umbral configurado en FCR
			THRE	1	RW	1’b0	uart_top.regs.ier[1]	Habilita “cola de transmisión vacía”.
			RLS	2	RW	1’b0	uart_top.regs.ier[2]	Habilita “error de linea”. Cuando se activa bits de error del LSR
			MS	3	RW	1’b0	uart_top.regs.ier[3]	Habilita “cambió una señal de modem”
			N/A	7:4	-	4’h0	-	

Registro	Dirección	DLAB	Campo	Bits	Acceso	Reset	Ruta de puerta trasera	Función
IIR	2	0 y 1	IP	0	RO	1b'1	uart_top.regs.iir[0]	0 = interrupción pendiente, 1 = No hay interrupción
			II	3:1	RO	3b'000	uart_top.regs.iir[3:1]	Identifica la causa de mayor prioridad (011 = error de línea, 010 = datos disponibles, 110 = timeout, 001 = cola TX vacía, 000=modem)
			N/A	7:4	-	-	-	
			FIFOEN	7:6	RO	2'b11	***	Constante 11, solo por compatibilidad con el chip original
FCR	2	0 y 1	N/A	0	WO	-	-	
			RFR	1	WO	1'b0	uart_top.regs.rx_reset	Escribir 1 limpia la cola de recepción. Autoborrable, dura un ciclo
			TFR	2	WO	1'b0	uart_top.regs.tx_reset	Escribir 1 limpia la cola de transmisión. Autoborrable
			N/A	5:3	WO	-	-	
			TL	7:6	WO	2'b11	uart_top.regs.fcr	Umbral de la cola de recepción 00=1, 01=4, 10=8, 11=14
MCR	4	0 y 1	DTR	0	WO	1'b0	uart_top.regs.mcr[0]	Controla la salida "terminal listo"
			RTS	1	WO	1'b0	uart_top.regs.mcr[1]	Controla la salida "petición de envío"
			OUT1	2	WO	1'b0	uart_top.regs.mcr[2]	Salida auxiliar. En loopback se conecta al indicador de llamada
			OUT2	3	WO	1'b0	uart_top.regs.mcr[3]	Salida auxiliar. En loopback se conecta a detección de portadora
			LB	4	WO	1'b0	uart_top.regs.mcr[4]	Modo espejo: Conecta la salida del transmisor a la entrada del receptor y fija stx_pad_o en 1
			N/A	7:5	-	-	-	
LSR	5	0 y 1	DR	0	RO	1'b0	uart_top.regs.lsr0r	Hay al menos un carácter en la cola de recepción
			OE	1	RO	1'b0	uart_top.regs.lsr1r	Overflow, llegó un carácter con la cola llena
			PE	2	RO	1'b0	uart_top.regs.lsr2r	El carácter en la cima de la cola tiene error de paridad
			FE	3	RO	1'b0	uart_top.regs.lsr3r	El carácter en la cima no tenía bit de parada válido
			BI	4	RO	1'b0	uart_top.regs.lsr4r	El carácter en la cima fue una condición de ruptura
			TFE	5	RO	1'b1	uart_top.regs.lsr5r	La cola de transmisión está vacía
			TE	6	RO	1'b1	uart_top.regs.lsr6r	Cola y registro de desplazamiento vacíos
			EI	7	RO	1'b0	uart_top.regs.lsr7r	Hay algún error en cualquier casilla de la cola, o hubo overflow
MSR	6	0 y 1	DCTS	0	RO	1'b0	uart_top.regs.msr[0]	"Libre para enviar" cambió desde la última lectura

Registro	Dirección	DLAB	Campo	Bits	Acceso	Reset	Ruta de puerta trasera	Función
			DDSR	1	RO	1’b0	uart_top.regs.msr[1]	“Modem listo” cambió
			TERI	2	RO	1’b0	uart_top.regs.msr[2]	El indicador de llamada cambió
			DDCD	3	RO	1’b0	uart_top.regs.msr[3]	“Portadora detectada” cambió
			CCTS	4	RO	1’b0	uart_top.regs.msr[4]	Pin “libre para enviar”
			CDSR	5	RO	1’b0	uart_top.regs.msr[5]	Pin “modem listo”
			CRI	6	RO	1’b0	uart_top.regs.msr[6]	Pin “indicador de llamada”
			CDCD	7	RO	1’b0	uart_top.regs.msr[7]	Pin “Portadora detectada”
SR	7	0 y 1	SCRATCH	7:0	RW	8’h00	Uart_top.regs.scratch	Registro de uso libre, escribe un valor y lo escribe igual. Existe para que el software pueda sondear la presencia del chip
DL1	0	1	DL1	7:0	RW	8’h00	uart_top.regs.dl[7:0]	Las dos mitades del DL, fija la velocidad serial. El valor del reloj del sistema dividido entre 16 veces el baudrate deseado. Escribir DL1 recarga el contador, escribir DL2 no. ESCRIBIR DL2 PRIMERO
DL2	1	1	DL2	7:0	RW	8’h00	uart_top.regs.dl[15:8]	
DBG1	8		LSR_V	7:0	RO	8’h60	uart_top.regs.lsr	Espejo no intrusivo de LSR
			IER_V	11:8	RO	4’h0	uart_top.regs.ier	Espejo no intrusivo de IER
			IIR_V	15:12	RO	4’h1	uart_top.regs.iir	Espejo no intrusivo de IIR
			LCR_V	23:16	RO	8’h03	uart_top.regs.lcr	Espejo no intrusivo de LCR
			MSR_V	31:24	RO	8’h00	uart_top.regs.msr	Espejo no intrusivo de MSR
DBG2	12		TSTATE	2:0	RO	3’h0	uart_top.regs.tstate	Estado de la maquina del transmisor
			TF_COUNT	7:3	RO	5’h00	uart_top.regs.tf_count	Cantidad de caracteres en la cola de transmisión
			RSTATE	11:8	RO	4’h0	uart_top.regs.rstate	Estado de la máquina del receptor
			RF_COUNT	16:12	RO	5’h00	uart_top.regs.rf_count	Cantidad de caracterres en la cola de recepción
			MCR_V	18:17	RO	5’h00	uart_top.regs.mcr	Copias del MCR
			FCR_V	23:19	RO	2’b11	uart_top.regs.fcr	Copias de FCR
			N/A	31	RO	8’h00	uart_top.regs	

12.2 Decisiones de modelado

Tema	Decisión	Justificación
Registros con dirección compartida (DLAB)	Se harán dos mapas de registros, uno llamada map_dlab0(RBR, THR, IER + IIR, FCR, LCR, MCR, LSR, MSR, SR, DBG1, DBG2) y otro llamado map_dlab1(DL1, DL2 + IIR, FCR, LCR, MCR, LSR, MSR, SR, DBG1, DBG2). Habrá un task set_dlab que escriba el bit de DLAB (el LCR [7]) antes de cambiar de mapa	Los mapas de UVM no permiten que haya registros con la misma dirección, así que se hacen dos mapas en un mismo reg_block. Es importante saber que si permite que haya registros que compartan misma dirección siempre y cuando uno solo escriba y el otro solo lea, por esto es que THR y RBR están en un mismo mapa a pesar de compartir dirección.
Predictor y adaptador: conexión y política de predicción	El adaptador separa la dirección para obtener el wb_sel_i(2LSB) y wb_addr_i (3MSB). Se utiliza predicción explicita donde se apaga set_auto_predict y set_check_read, habrá un uvm_reg_predictor conectado al puerto del monitor Wishbone, adaptador y mapa.	La dirección del registro que se quiera acceder de wishbone se construye de la forma wb_adr_int = {wb_adr_is[4:2], wb_adr_int_lsb}, donde el wb_adr_int_lsb se obtiene del número de case y no de la dirección. El ambiente también configura el DUT con secuencias wishbone que no pasan el RAL (como un reset de FIFO a través de FCR).
Efectos laterales (lectura que limpia, FIFOs)	RBR, THR, LSR, IIR, MSR, DBG1 y DBG2 se declaran volátiles y con UVM_NO_CHECK al leer, el valor lo predice el scoreboard. Los bits de LSR[1:4], LSR[7] y MSR[3:0] se modelan como RO con un callback que los pone en 0 si la lectura fue por el map_dlab0 Toda lectura y escritura (RBR y THR) lo trabaja el modelo de referencia (este si permite trabajarlas como FIFO) FCR[1,2] se declaran WO y el predictor los deja en 0 despues de cada escritura Escribir DL1 tiene un callback que le dice al sb que el contador del divisor se recargó Para observar LSR, IIR, MSR sin limpiar se usa DGB1	En el RTL, leer RBR saca un dato de la FIFO RX, leer LSR limpia los errores y el pendiente RLS, leer MSR limpia los deltas y el pendiente de módem, leer IIR limpia THRE y escribir THR llena la FIFO TX y limpia THRE. Además, LSR, IIR, MSR y RBR cambian por hardware sin mediar transacción. Si el RAL comparara estos registros contra su espejo, daría falsos errores. Las limpiezas solo ocurren con DLAB = 0. Por eso se condicionan al mapa en un callback y no se usa el acceso RC, que limpiaría también al leer por map_dlab1. DBG1 refleja MSR, LCR, IIR, IER y LSR sin efectos laterales, así que es la forma no intrusiva de comparar esos registros contra el modelo.
Secuencias integradas que se correrán y exclusiones	Se puede correr uvm_reg_hw_reset_seq y uvm_reg_bit_bash_seq sobre SR, LCR, IER, uvm_reg_access_seq sobre los mismos tres y uvm_reg_shared_access_seq sobre SR y LCR. DL1 y DL2 se verifica con una secuencia propia que escribe MSB antes de LSB	FCR Y MCR no aparecen en el case de lectura de las líneas por lo que devuelven cero. THR Y RBR son ventanas de las FIFOS y cualquier barrido pierde o inyecta datos, leer IIR LSR o MSR limpia los pendientes de interrupción y bits de error. Hardware mueve IIR, LSR, MSR, DBG1 y DBG2 sin mediar transacción, por lo que el hw_reset fallaría (valor cambia entre reset y lectura). DL1 y DL2 exigen MSB antes de LSB. LCR e IER corren con trafico apagado porque escribir valores arbitrarios en LCR altera toc_value y block_value.

13. Modelo de cobertura funcional

Covergroup	Dónde se muestrea	Coverpoints	Cruces	Bins ilegales o ignorados (justificación)
cg_line_cfg	En el modelo de referencia al completar un carácter tanto en TX o RX, cuando se	cp_wls (5, 6, 7, 8) cp_stb (1, 1.5/2 dependiendo del largo) cp_parity (none, par, impar, stick0, stick1) cp_bc (break)	cp_wls × cp_dir cp_stb × cp_wls cp_parity × cp_dir cp_dl × cp_dl_wr_order	N/A

Covergroup	Dónde se muestrea	Coverpoints	Cruces	Bins ilegales o ignorados (justificación)
	escribe formato de LCR y al terminar de escribir DLL Y DLM	cp_dir(TX, RX) cp_dl (zero, fast, slow) cp_dl_wr_order (DLL PRIMERO, DLM PRIMERO) cp_dl_zero (DL=0)		
cg_fifo	En el modelo, en cada ciclo con push, pop o limpieza en cualquiera de las dos FIFOs.	cp_rf_level (0, 1 a 15, 16) cp_tl (1, 4, 8, 14) cp_tf_level (0, 1 a 15, 16) cp_fifo_clear (RX, TX) cp_rbr_read (si, no) cp_rx_push (si, no)	cp_rf_level × cp_tl cp_rbr_read × cp_rx_push	Ilegal: rf_count o tf_count mayor que 16. Los contadores son de 5 bits, pero la profundidad es 16. En cp_rbr_read × cp_rx_push, el bin SI × SI es la lectura de RBR y la llegada de un carácter en el mismo ciclo que busca CE-07. NO × NO corresponde a un ciclo con solo limpieza.
cg_int	En el modelo de interrupciones, cuando sube int_o y en cada lectura de IIR, cada vez que se limpia una causa pendiente y en cada escritura de IER	cp_iir_id: (rls, rda, timeout, thre, modem_status, none) cp_pending_set (única, varias) interrupciones pendientes cp_rx_data_ready_pending (si, no) cp_tl (1, 4, 8, 14) cp_loopback_active (encendido, apagado) cp_clear_by (thr_write, iir_read, lsr_read, msr_read, rbr_read) cp_ier (combinaciones de ier[3:0])	cp_iir_id × cp_pending_set cp_iir_id[TIMEOUT] × cp_tl cp_iir_id[RDA] × cp_tl cp_iir_id[RLS] × cp_rx_data_ready_pending[SI] cp_iir_id[MODEM_STATUS] × cp_loopback_active	Ilegal: IIR[0] = 0 con IIR[3:1] = 100, 101 o 111 (códigos que no existen). Ignorado en cp_iir_id × cp_pending_set: MODEM_STATUS con VARIAS (es la causa de menor prioridad) y NONE con ambos bins (no hay causas pendientes).
cg_err	En el modelo al insertar cada carácter en la FIFO de RX y en cada carácter que manda el agente serie	cp_pe: SI cp_fe: (si, no) cp_break: (si, no) cp_oe: (si) cp_parity (none, par, impar, stick0, stick1) cp_tl (1,4,8,14) cp_baud_dev_high(0 a 4%, 4 a 5%, 6+) cp_glitch(start, dato)	cp_pe × cp_parity cp_fe[SI] × cp_parity cp_break[SI] × cp_parity cp_oe × cp_tl cp_break × cp_fe	Ignorado: cp_pe con cp_parity = NONE. Sin paridad no hay bit que corromper. Ignorado en cp_break × cp_fe: SI × SI. El RTL inserta el break como {8'b0, 3'b100}, con FE = 0, así que la combinación no existe (confirmado en simulación).
cg_modem	En el modelo de MSR, cuando hay cambios en cts, dsr, ri, o dcd, escrituras de MCR y al completar el carácter	cp_delta: (DCTS, DDSR, TERI, DDCD) cp_ri_edge (subida, bajada) cp_loopback_active: OFF, ON cp_mcr_bits (DTR, RTS, OUT1 y OUT2) todos 1 y 0 cp_wls(5,6,7,8)	cp_delta[TERI] × cp_ri_edge cp_loopback_active × cp_delta cp_loopback_active × cp_wls	N/A
cg_regs	En cada transacción Wishbone con ack. El modelo le agrega a la transacción DLAB y si había errores pendientes.	cp_access (0, 1 a 3, 3+ ciclos) cp_addr (1 bin por reg + DGB1 y DGG2) cp_sel (one_hot, full, illegal) cp_rw (read, write) cp_dlab: (0, 1) cp_lsr_read (si)	cp_sel × cp_addr cp_addr × cp_dlab × cp_rw cp_lsr_read × cp_err_pending	Ignorado en cp_sel × cp_addr: full e illegal con las direcciones 1, 2, 3, 5, 6 y 7, porque con esos sel la dirección interna siempre termina en 00. También one_hot con 8 o 12: los registros de depuración solo responden con 4'b1111 y con otro sel devuelven 0. Las direcciones 9 a 11 y 13 a 15 no están en cp_addr: no hay

Covergroup	Dónde se muestrea	Coverpoints	Cruces	Bins ilegales o ignorados (justificación)
		cp_err_pending (si, no)		registro. cp_addr × cp_dlab × cp_rw deja fuera 8 y 12: no dependen de DLAB y no se escriben.
cg_trafico	En el modelo al terminar escrituras de THR y al leer carácter de RBR	cp_tx_burst_len: (1, 2 a 15, 16, OVERFLOW) cp_tx_write_attempt (si) cp_dl_zero (si, no) cp_wls (5,6,7,8) cp_rx_gap (0, 1 a 5, 6 a 20, más de 20)	cp_tx_burst_len × cp_wls cp_rx_gap × cp_wls cp_dl_zero × cp_tx_write_attempt	
cg_rst	En cada flanco de subida de wb_rst_i después del reset inicial.	cp_during (idle, tx_activo, rx_activo, fifos no vacías).	Ninguno	N/A

14. Cobertura de código

Métrica	Jerarquía incluida	Meta	Exclusiones previstas y justificación
Línea	uart_top y toda su descendencia: uart_wb, uart_regs, uart_transmitter, uart_receiver, uart_tfifo, uart_rfifo, raminfr, uart_sync_flops y uart_debug_if	≥ 95 %	En UART receiver (línea 417) y uart_transmitter (línea 341) ambas son comentadas por el autor como inalcanzables. El registro de estado tiene más bits de los que la FSM usa
Condición	uart_top y toda su descendencia: uart_wb, uart_regs, uart_transmitter, uart_receiver, uart_tfifo, uart_rfifo, raminfr, uart_sync_flops y uart_debug_if	≥ 95 %	Condiciones sobre stage1_rst y stage1_clk_en en el uart_sync_flops.sv (líneas 116 y 118), la instanciación de uart_regs.sv (línea 382-390) les asigna 1’b0 y 1’b1, por lo que dos de las tres ramas del flip flop nunca se ejercitan.
Rama	uart_top y toda su descendencia: uart_wb, uart_regs, uart_transmitter, uart_receiver, uart_tfifo, uart_rfifo, raminfr, uart_sync_flops y uart_debug_if	≥ 95 %	Todos los caminos son alcanzables
FSM	uart_top y toda su descendencia: uart_wb, uart_regs, uart_transmitter, uart_receiver, uart_tfifo, uart_rfifo, raminfr, uart_sync_flops y uart_debug_if	≥ 95 %	rstate en uart_receiver.v (líneas 262-272) es de 4 bits y usa 11 estados, por lo que del 11 al 15 no se usan. En uart_transmitter tsate es de 3 bits y usa 6 estados, por lo que soban 6 y 7 (líneas 203-208). Wbstate en uart_wb.v (líneas 201-222) es de 2 bits y la máquina pasa por 0,1,2 pero nunca llega a 3.
Toggle	uart_top y toda su descendencia: uart_wb, uart_regs, uart_transmitter, uart_receiver, uart_tfifo, uart_rfifo, raminfr, uart_sync_flops y uart_debug_if	≥ 95 %	Todos los bits conmutan con la prueba general + casos esquina

15. Criterios de cierre

Criterio	Meta	Cómo se mide
Cobertura funcional	≥ 95 %	Fusión de los covergroups de la sección 13 sobre la regresión completa
Cada métrica de cobertura de código	≥ 95 %	Banderas de cobertura de VCS

Criterio	Meta	Cómo se mide
UVM_ERROR en la regresión	0, salvo errores del DUT documentados	Salida del uvm_report_server
Aserciones con cover alcanzado	100 %	Reporte de cobertura de las aserciones
Casos de esquina	Todos alcanzan su objetivo	Reporte final verificando que la condición se haya cumplido

16. Riesgos y mitigación

Riesgo	Impacto	Mitigación
Adaptador deriva mal el wb_sel_i	Bloquea cualquier prueba que configure el DUT por registros	Pruebas sobre el DUT de leer SR por bus y confirmar por puerta trasera
Un error del ambiente se reporte como error del DUT	Credibilidad durante el reporte	Todo UVM_ERROR se reproduce con su semilla y se clasifica antes de documentarlo.
Conocimiento solo sobre los componentes que cada uno trabajó	Afectación en la entrevista	Revisiones cruzadas durante las semanas

17. Distribución del trabajo

Componente o tarea	Responsable principal	Revisor(es) (otro integrante)	Semana
Secciones del testplan: 1, 2, 3, 4, 5, 6 y 7.	Sergio Salazar Núñez	Juan Pablo Elizondo Espinoza Josué Campos Herrera	2
Secciones del testplan: 8, 9, 10, 11 y 17.	Juan Pablo Elizondo Espinoza	Sergio Salazar Núñez Josué Campos Herrera	2
Secciones del testplan: 12, 13, 14, 15 y 16.	Josué Campos Herrera	Sergio Salazar Núñez Juan Pablo Elizondo Espinoza	2
Interfaz Wishbone	Juan Pablo Elizondo Espinoza	Sergio Salazar Núñez Josué Campos Herrera	4
Interfaz Serie	Sergio Salazar Núñez	Juan Pablo Elizondo Espinoza Josué Campos Herrera	4
Interfaz Módem	Josué Campos Herrera	Sergio Salazar Núñez Juan Pablo Elizondo Espinoza	4
Aserciones (Wishbone)	Juan Pablo Elizondo Espinoza	Sergio Salazar Núñez Josué Campos Herrera	4
Aserciones (Serie)	Sergio Salazar Núñez	Juan Pablo Elizondo Espinoza Josué Campos Herrera	4
Aserciones (Módem)	Josué Campos Herrera	Sergio Salazar Núñez Juan Pablo Elizondo Espinoza	4
Agente Wishbone (junto con sus componentes internos)	Juan Pablo Elizondo Espinoza	Sergio Salazar Núñez	4

Componente o tarea	Responsable principal	Revisor(es) (otro integrante)	Semana
		Josué Campos Herrera	
Agente Serie (junto con sus componentes internos)	Sergio Salazar Núñez	Juan Pablo Elizondo Espinoza Josué Campos Herrera	4
Agente Módem (junto con sus componentes internos)	Josué Campos Herrera	Sergio Salazar Núñez Juan Pablo Elizondo Espinoza	4
Infraestructura RAL (modelo, predictor y adaptador)	Juan Pablo Elizondo Espinoza	Sergio Salazar Núñez Josué Campos Herrera	4
Scoreboard	Josué Campos Herrera	Sergio Salazar Núñez Juan Pablo Elizondo Espinoza	4
Secuenciador Virtual	Sergio Salazar Núñez	Juan Pablo Elizondo Espinoza Josué Campos Herrera	4
Recolector de cobertura	Juan Pablo Elizondo Espinoza	Sergio Salazar Núñez Josué Campos Herrera	5
Ambiente	En conjunto	En conjunto	4
Test base	En conjunto	En conjunto	4
Top tb	En conjunto	En conjunto	4
wb_agent_cfg	Juan Pablo Elizondo Espinoza	Sergio Salazar Núñez Josué Campos Herrera	4
serial_agent_cfg	Sergio Salazar Núñez	Juan Pablo Elizondo Espinoza Josué Campos Herrera	4
mod_agent_cfg	Josué Campos Herrera	Sergio Salazar Núñez Juan Pablo Elizondo Espinoza	4
uart_virtual_seqr_cfg	Sergio Salazar Núñez	Juan Pablo Elizondo Espinoza Josué Campos Herrera	4
uart_scoreboard_cfg	Josué Campos Herrera	Sergio Salazar Núñez Juan Pablo Elizondo Espinoza	4
uart_reg_block_cfg	Juan Pablo Elizondo Espinoza	Sergio Salazar Núñez Josué Campos Herrera	4
uart_cov_collector_cfg	Juan Pablo Elizondo Espinoza	Sergio Salazar Núñez Josué Campos Herrera	5
uart_env_cfg	En conjunto	En conjunto	4
Análisis de huecos de cobertura	En conjunto	En conjunto	6